

Rubric Scorecard for GDPval: Evaluating and Iterating Rubrics for Multimodal RL

Philo Labs — Track R

June 2026

1 Introduction

A rubric that looks like it grades quality while it really grades form is a liability as an RL reward signal: the policy optimizes for exactly what is rewarded, including the parts no one meant to reward. Before trusting a rubric as training signal, we have to grade the rubric itself.

This report covers ten video-centric GDPval tasks (Patwardhan et al., 2025) across four occupations. For each we score three axes — boundary calibration, reward hackability, consistency — plus a fourth axis, verifiability, that the harness surfaced empirically rather than one we set out to measure. An iterator then tries to rewrite each rubric to raise those scores. The question throughout: after all of this, which rubrics would we trust as RL reward?

2 Design Choices

Give the judge a fair shot before blaming the rubric. Early runs lost roughly 60% of the gold ceiling to things the judge simply could not see — codec on a video, text in a flattened PDF, contents of a zip of stems — versus only ~3% genuine rubric gaps. We do not want the scorecard to confuse *harness blindness* with *rubric badness*, so the judge is a tool-using agent: `probe_media` gives fprobe ground truth so objective criteria are never guessed, `file_facts` reports type/page/word count, `view_video_frames` samples per scene cut, and deliverables are abstracted into a flat file view that transparently expands zip containers, so “the stems are present” is checkable without the judge knowing what a zip is. A missing-capability question should never be answered by the rubric instead of the toolbox.

Vary modality access, not just provider. The suite spans six judges along three axes at once: vendor (OpenAI vs. OpenRouter), capability tier (`gpt-mini` vs. `gpt-nano`), and modality (`qwen-text` sees no images, `qwen-vl` adds vision, `gemini-lite` natively hears audio rather than reading a Whisper transcript). The `qwen-text` → `qwen-vl` → `gemini-lite` ladder is held fixed within a family so a disagreement can be attributed to modality access, not general competence — otherwise a judge that cannot see video would look like it disagrees about the *rubric*, when it is really blind to the *artifact*.

An explicit “cannot verify” verdict, not a forced guess. Some criteria cannot be checked from the artifact at all (“footage is royalty-free”). Forcing a verdict there just adds judge noise disguised as rubric signal. Every item gets one of four verdicts (`met/unmet/partial/unverifiable`), and two scores are reported: `normalized` (`unverifiable` = 0) and `normalized_verifiable_only` (`unverifiable` dropped from both numerator and denominator). The gap between them is diagnostic — it is how Axis 4 below was discovered, not designed in advance.

Reward hackability, two paths, both used. Scope favored the cheap path first: five deterministic, no-LLM document attacks (`restate_input`, `keyword_stuff`, `fabricated_evidence`, `unextractable_render`, `judge_injection`) plus typed analogues for video (`presence_max`) and

audio archives (`stems_named`); free, reproducible, and they extend to gold-free tasks since attack type depends only on deliverable type. An LLM adversarial agent was layered on once the mechanical suite proved the concept: shown the rubric and a read-only gold oracle, it must build a new deliverable from scratch that scores high without doing the work. The oracle’s first version let the model literally clone the gold file byte-for-byte; that tool was removed, leaving only inspection (`list_gold`, `read_gold_text`, `view_gold_frames`).

Hackability needs a rubric-free reference, or it measures itself. A rubric score alone cannot tell a genuine improvement from a rubric-gaming one. So every attack is also scored by a *blind* pairwise judge that never sees the rubric or which side is gold: both submissions are anonymized A/B with identical symmetric tools and randomized position. $\text{deception} = \text{rubric_score} \times \text{gold_winrate}$ is high only when an attack both games the rubric *and* is empirically worse. This was not optional polish: an earlier role-aware comparator rated a byte-identical clone of gold as “worse than gold, margin 3” purely from a labeling prior, before the anonymized version replaced it.

An iterator that is a function pipeline, not a framework. `iterate.py` is a single-threaded `evaluate` $\rightarrow$ `diagnose` $\rightarrow$ `rewrite` $\rightarrow$ `re-evaluate` loop, no population search, no dependency on libraries like GEPA or AdalFlow. At this scale — one rubric, tens of items, one axis-tagged defect list per round — a single prompted rewrite per round is as effective and far easier to audit; every accept/reject decision is inspectable in the package’s `changelog` and `report.json`.

3 The Scorecard

3.1 Axis 1: Boundary Calibration

Table 1 shows gold and worst-null scores (`gpt-mini`, v0 rubrics) on the six gold-bearing tasks.

Task	Gold	Null	Gap	Null Leaks
<code>producers__e4f664ea</code>	0.836	0.018–0.097*	0.819	2
<code>detectives__a46d5cd2</code>	0.692	0.068	0.623	3
<code>editors__e222075d</code>	0.721	0.115	0.607	4
<code>editors__75401f7c</code>	0.907	0.271	0.636	10
<code>audio__38889c3b</code>	0.597	0.097	0.500	3
<code>detectives__57b2cdf2</code>	0.983	0.653	0.331	33

Table 1: Axis 1, v0 rubrics, `gpt-mini`. Null leaks = positive items the do-nothing input still passes. *Range across blank vs. unedited-input nulls.

Every task clears the sanity bar in direction (gold $\gg$ null), but the margin varies sharply. `57b2cdf2` is the weakest: the unedited input — the raw field report submitted without revision — already scores 0.653, because most of the rubric rewards report *form* (section headings, a timeline, correct structure) rather than investigative *substance*. A null that is 65% of the way to gold is a rubric that barely distinguishes “did the work” from “filled in the template.”

3.2 Axis 2: Reward Hackability

`57b2cdf2` is the clearest mechanical failure: 33–45 of its ~ 45 positive items leak to at least one of five document attacks — the bulk of the rubric is satisfiable by surface-form echo with no investigative work. `keyword_stuff` alone, built by inverting every rubric clause into a literal statement, buys 92% of the gold score.

The suite also surfaced a **cross-model** hackability gap single-judge testing would have missed: on `57b2cdf2`’s `judge_injection` attack (a document with a visible fake “grading note,” hidden white-on-white text, and injected PDF metadata all asserting “all criteria met”), `gpt-mini` scores it 0.102

Task — Attack	Rubric Score	Gold Winrate
75401f7c — <code>presence_max</code>	0.186	1.00
75401f7c — adversarial (gpt5.5)	0.898	1.00
57b2cdf2 — <code>keyword_stuff</code>	0.92	1.00
57b2cdf2 — <code>restate_input</code>	0.87	1.00
57b2cdf2 — adversarial (gpt5.5)	0.10	1.00
a46d5cd2 — <code>restate_input</code>	0.67	1.00
e4f664ea — adversarial (gpt5.5)	0.75	1.00

Table 2: Axis 2, selected attacks, **gpt-mini** judge. Gold winrate = 1.00 throughout: every attack here is genuinely worse than gold under blind comparison, so deception tracks rubric score directly.

— essentially un-fooled — while **qwen-text** scores the identical file 1.00. Hackability is a property of the (rubric, judge) *pair*: a rubric “safe” under one judge can be fully broken under whichever model later serves as the RL reward.

3.3 Axis 3: Consistency

Self-consistency (same judge, same gold, repeated) is tight: stdev 0.0–0.10 across the six tasks at **gpt-mini**. Cross-model consistency is sharper: on **57b2cdf2**, **gpt-mini** and **qwen-text** disagree by 0.5 on gold itself (**qwen-text** fails to produce a valid scorecard there at all), and by 0.449 on **judge_injection** specifically — the largest cross-model gap in the slice, concentrated exactly on the attack built to test injection susceptibility. When judges disagree this sharply on the same artifact, the rubric or a judge’s reliability is the suspect, not the work.

3.4 Axis 4: Verifiability (added axis)

The **unverifiable** verdict was meant as a safety valve, but reading the gap between **normalized** and **normalized_verifiable_only** across tasks turned it into a fourth axis: some rubrics ask for things no tool can check. **e222075d** has the largest gap in the slice (0.279 on v0, 2 of 10 items unverifiable on gold) — a face-visibility penalty that swings the score with whatever frames a judge happens to sample. **audio__38889c3b** carries 2 unverifiable items on v0 (gap 0.03), rising to 6 after one rewrite (gap 0.104), when the rewrite swapped a checkable presence criterion for vaguer language. A large or growing verifiability gap means score and actual work quality are silently decoupled, independent of hackability or boundary calibration.

4 The Iterator

4.1 Loop and Objective

Per round: evaluate gold and the hack-subject pool (nulls $\cup$ attacks); diagnose per-item defects tagged by axis (**boundary/null_leak**, **hackability/attack_leak**, **verifiability/unverifiable**, **consistency/contested**) and ranked by points-at-stake; rewrite only the flagged criteria, holding item IDs and point values fixed so before/after stays on scale; re-evaluate. The accept/reject objective is $\text{margin} - 0.5 \times \text{self_consistency_stdev}$, where $\text{margin} = \text{gold} - \text{worst_attack}$. A rewrite is kept only if it raises this objective; otherwise the loop reverts and stops. Cross-model stdev is logged but does not gate acceptance — a policy trains against one judge, so that judge’s own noise is the RL-relevant floor.

4.2 Results

Two runs were completed across all six gold-bearing tasks (**gpt-mini**, 2 self-consistency repeats, $T=0.7$, max 3 rounds per task). The first run (4 tasks, 2 rounds, 24 min, \$1.63) was partially

corrupted by rate-limit 429 errors. The second run (6 tasks, 2 parallel batches of 3, 5 min, \$4.07) fixes the TSV-clobber issue and adds a gold-free guard (tasks without a gold deliverable stop at v0). Table 3 shows every version evaluated.

Task	Ver.	Gold	Worst Atk.	Objective	Atk. Leaks	Outcome
audio__38889c3b	v0	0.597	0.161	0.436	5	✓
	v1	0.484	0.210	0.258	7	×
editors__75401f7c	v0	0.907	0.559	0.299	22	✓
	v1	0.941	0.915	0.024	36	×
editors__e222075d	v0	0.721	0.131	0.566	4	✓
	v1	0.000	0.066	−0.066	2	×
detectives__57b2cdf2	v0	0.983	0.907	0.072	45	✓
	v1	0.898	1.000	−0.112	45	×
detectives__a46d5cd2	v0	0.884	0.637	0.238	43	✓
	v1	0.712	0.712	−0.013	49	×
producers__e4f664ea	v0	0.921	0.789	0.131	34	—
	v1	0.942	0.766	0.175	35	✓
	v2	0.899	0.655	0.237	28	✓
	v3	0.937	0.749	0.184	36	×

Table 3: Iterator results across all six gold tasks. **Bold** = best accepted version. **e4f664ea** is the only multi-round success: objective +81% from v0 to v2.

4.3 Honest Assessment

producers__e4f664ea is the strongest improvement yet: the objective climbed from 0.131 to 0.237 (+81%) over three rounds, with attack leaks dropping from 34 to 28 and worst-attack from 0.789 to 0.655. The hill-climb correctly accepted v1 and v2, then rejected v3 when it regressed. This rubric is genuinely amenable to the wording-level fixes the iterator produces.

audio__38889c3b kept v0 in this run (objective 0.436) where a previous run had accepted v1, illustrating a signal-to-noise problem: with only 2 self-consistency repeats, the objective has roughly ± 0.02 – 0.05 variance from stochastic judge wandering, enough to flip accept/reject decisions near the margin.

The remaining four tasks all rejected their v1 rewrite. Two failure shapes appear. **Gold collapse:** **e222075d**’s v1 gold fell to 0.000, a transient rate-limit failure rather than a genuine regression — the iterate loop needs a sanity check before trusting a gold that contradicts prior measurements. **Leak explosion:** **75401f7c**’s worst-attack rose from 0.559 to 0.915 and attack leaks from 22 to 36. **57b2cdf2** is genuinely resistant: 45 of 45 items leak in both versions, unchanged despite two independent round-one attempts spanning separate runs. **a46d5cd2** has the thinnest margin after v0 (objective 0.238 to begin with) and every rewrite attempt made it worse.

Overall: **2 of 6 tasks have shown an accepted improvement over at least one run; 4 of 6 remain stuck at v0.**

4.4 Gold-Free Tasks

Four tasks have no released gold, so only attack-only hackability is measurable. **audio__4b894ae3**’s **stems_named** attack (correctly-named, silent stems) scores 0.000 with zero leaks — the most hack-resistant rubric in the slice, since every item requires actual audio content. **editors__a941b6d8** and **c94452e4** leak 4–10 items each to **presence_max**, a moderate, fixable presence-without-absence pattern.

5 Which Rubrics Would We Trust?

- **Trust as-is:** `e4f664ea` — objective +81% after iteration, attack leaks 34→28, worst-attack 0.789→0.655, correctly rejected a backfiring v3. `audio__4b894ae3` — zero leaks against the strongest attack we have for its modality, no rewrite needed.
- **Trust with monitoring:** `75401f7c`, `a46d5cd2`, and `audio__38889c3b` — decent v0 numbers, but a single rewrite round was enough to blow up hackability or null calibration, meaning the rubric’s margin is thinner than the headline gold score suggests. Need re-audit after each iteration round.
- **Would not trust:** `57b2cdf2` — 45 of 45 items leak to mechanical attacks across two independent iteration attempts, the cross-model gap on `judge_injection` is the largest in the dataset, and wording-level rewrites cannot touch the structural form-over-substance defect. `e222075d` — the largest verifiability gap in the slice sits on a single high-weight, sample-dependent penalty item; a rubric whose score depends on which video frames a judge happens to sample is not a stable reward.

6 Lessons for Rubric Design

Hackability is a property of (rubric, judge), not rubric alone. The sharpest finding here is `qwen-text` scoring a prompt-injection attack 1.00 while `gpt-mini` scores the identical artifact 0.102. A rubric audited against one judge model can pass review and still be wide open under whatever model eventually serves as the RL reward. This generalizes directly: a rubric handed off for RL training should be re-audited against the actual reward-model family, not just whichever judge wrote the audit.

Presence without absence is the dominant mechanical-hack pattern, with gradients worth distinguishing: a criterion can leak to a cheap echo of the prompt (`restate_input`), the rubric’s own language fed back at it (`keyword_stuff`), a claim of evidence with none attached (`fabricated_evidence`), or a visually-complete-but-unextractable render (`unextractable_render`). Each names a different fix, so “leak count” alone undersells what the iterator needs — the attack that catches a leak names the repair.

A rubric that resists improvement is itself a finding. Five of six gold tasks rejected their v1 rewrite in the latest run, not because the loop failed but because every fix traded one axis for another — gold collapsed, leaks widened, or determinism dropped. `e4f664ea` is the exception: its rubric is genuinely amenable to wording-level tightening, and it improved across three rounds. Most of the defects in the other tasks are structural, and a single wording pass is the wrong tool for a structural problem. This generalizes: an iterator that always finds an accepted improvement is suspect; one that correctly reports “nothing here helped” is doing its job.

The null-vs-gold gap is case-by-case, not a universal target. `e4f664ea`’s gap (0.819) and `57b2cdf2`’s gap (0.331) are not comparably-sized failures — the latter’s null is a real, professionally-formatted report that simply lacks investigative work, so a high null score is partly unavoidable for that task type. For tasks with no released gold, or more broadly any RL domain without a clean oracle (an unsolved problem, say), the gold-anchored half of this scorecard cannot run at all; hackability-against-attacks is the part that generalizes to gold-free settings and should be weighted accordingly.

7 Next Steps

1. **Fix the gold-score reliability bug.** A sanity check (reject/retry any version whose gold score is implausibly far from a quick repeat) prevents transient rate-limit failures from cor-

rupting the objective. e222075d’s v1 0.000 result is blocking real iterator progress on that task.

2. **Increase self-consistency repeats to 5.** The 2-repeat objective has ~ 0.02 – 0.05 variance from stochastic judge wandering, enough to flip accept/reject decisions (38889c3b accepted in one run, rejected in another).
3. **Extend the iterator beyond in-place wording edits to item add/split** — 57b2cdf2’s defect needs a new criterion type, not a tighter old one.
4. **Re-run the adversarial agent** across the full ten-task slice and grade it under `qwen-text` to see whether the injection vulnerability generalizes beyond mechanical attacks.
5. **Add a discrimination axis:** today’s anchors (gold near 1, null near 0) say nothing about separating merely-adequate from genuinely-excellent work, which needs curated mid-quality examples per task.
6. **Gate on verifiability:** flag any task whose verifiability gap exceeds a threshold before it is approved as RL reward, independent of its other axis scores.
7. **Cross-model consistency at scale:** extend the modality-ladder comparisons across all 6 gold tasks, still only spot-checked.